

Logic toán học đơn giản và ứng dụng

Li Kaiwei

Khoa Khoa học và Công nghệ Máy tính, Đại học Thanh Hoa

Nguồn gốc: Simple mathematical logic and its applications

Mathematics / Simple mathematical logic and applications - Li Kaiwei

Abstract

Bài giảng giới thiệu các khái niệm cơ bản của logic mệnh đề: mệnh đề, biến mệnh đề, liên kết logic, công thức hợp lệ, công thức tương đương và dạng chuẩn. Từ đó tài liệu chuyển sang bài toán SAT, trường hợp đặc biệt 2-SAT, thuật toán DPLL, rồi mở rộng sang SMT với các miền như số nguyên, số thực, mảng và bit-vector. Phần cuối minh họa cách giải hệ tuyến tính trên bit-vector và các ứng dụng trong giải phương trình, sinh kiểm thử và phát hiện lỗi chương trình.

Contents

1	Logic mệnh đề	1
1.1	Mệnh đề và biến mệnh đề	1
1.2	Liên kết logic	1
1.3	Công thức hợp lệ	2
1.4	Phân loại công thức	2
2	Tương đương và dạng chuẩn	2
2.1	Công thức tương đương	2
2.2	Từ bảng chân lý đến công thức	3
2.3	Dạng chuẩn	3
3	Bài toán SAT	4
3.1	Định nghĩa	4
3.2	2-SAT	4
3.3	3-SAT và DPLL	4
4	SMT	5
4.1	Từ SAT đến SMT	5
4.2	Ví dụ giải hệ tuyến tính trên bit-vector	5
5	Ứng dụng	6
5.1	Giải phương trình	6
5.2	Phát hiện lỗi chương trình	6

1 Logic mệnh đề

1.1 Mệnh đề và biến mệnh đề

Một **mệnh đề** là một câu trần thuật có giá trị chân lý xác định: hoặc đúng, hoặc sai. Ví dụ “tuyết màu trắng” là một mệnh đề đúng; câu tự tham chiếu kiểu “điều tôi đang nói là sai” cho thấy nếu phát biểu không có giá trị chân lý ổn định thì cần xử lý cẩn thận hơn.

Khi hình thức hóa, ta dùng ký hiệu như P, Q, R để biểu diễn các mệnh đề. Một **biến mệnh đề** có thể nhận giá trị T hoặc F. Một mệnh đề phức hợp có thể phân tách thành nhiều mệnh đề đơn. Ví dụ

$$P : \text{ “tuyết màu trắng và } 1 + 1 = 2\text{”}$$

có thể tách thành

$$R : \text{ “tuyết màu trắng”, } \quad S : \text{ “} 1 + 1 = 2\text{”}.$$

1.2 Liên kết logic

Các liên kết thường dùng gồm phủ định $\neg$, hội $\wedge$, tuyển $\vee$, kéo theo $\rightarrow$, và tương đương $\leftrightarrow$. Bảng chân lý cơ bản:

p	$\neg p$	p	q	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
F	T	F	F	F	F	T	T
F	T	F	T	F	T	T	F
T	F	T	F	F	T	F	F
T	F	T	T	T	T	T	T

Trong công thức $p \rightarrow q$, giá trị sai chỉ xuất hiện khi tiền đề đúng nhưng kết luận sai. Điều này phản ánh cách logic hình thức hóa “nếu p thì q ”, không nhất thiết là quan hệ nhân quả trong đời sống.

1.3 Công thức hợp lệ

Một **công thức hợp lệ** là chuỗi ký hiệu được tạo từ biến mệnh đề và các liên kết theo quy tắc cú pháp:

- Mỗi biến mệnh đề là một công thức hợp lệ.
- Nếu A là công thức hợp lệ thì $\neg A$ cũng là công thức hợp lệ.
- Nếu A, B là công thức hợp lệ thì $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$, và $(A \leftrightarrow B)$ đều là công thức hợp lệ.
- Chỉ các chuỗi thu được sau hữu hạn lần dùng ba quy tắc trên mới là công thức hợp lệ.

Ví dụ:

$$(p \wedge (p \rightarrow q)) \rightarrow q$$

là một công thức hợp lệ. Một biểu thức lập trình như `if A then B else C` cũng có thể được biểu diễn bằng công thức logic, chẳng hạn

$$(A \wedge B) \vee (\neg A \wedge C).$$

1.4 Phân loại công thức

Với một diễn giải I , mỗi biến mệnh đề được gán giá trị đúng hoặc sai. Một công thức được gọi là:

- hằng đúng** nếu nó đúng dưới mọi diễn giải;
- khả thỏa** nếu nó đúng dưới ít nhất một diễn giải;

- **mâu thuẫn** nếu nó sai dưới mọi diễn giải.

Các quan hệ cơ bản:

$$A \text{ hằng đúng} \iff \neg A \text{ hằng sai},$$

$$A \text{ khả thỏa} \iff \neg A \text{ không hằng đúng},$$

$$A \text{ không khả thỏa} \iff A \text{ hằng sai}.$$

2 Tương đương và dạng chuẩn

2.1 Công thức tương đương

Hai công thức A, B được gọi là **tương đương**, ký hiệu $A = B$, nếu chúng có cùng giá trị chân lý dưới mọi diễn giải của các biến xuất hiện trong chúng. Khi đó $A = B$ khi và chỉ khi $A \leftrightarrow B$ là hằng đúng. Ký hiệu $=$ ở đây không phải là một liên kết logic mới, mà là quan hệ giữa hai công thức. Quan hệ này có tính phản xạ, đối xứng và bắc cầu.

Một số luật tương đương thường dùng:

Phủ định kép: $\neg\neg P = P,$

Giao hoán: $P \vee Q = Q \vee P, \quad P \wedge Q = Q \wedge P,$

Kết hợp: $(P \vee Q) \vee R = P \vee (Q \vee R),$
 $(P \wedge Q) \wedge R = P \wedge (Q \wedge R),$

Phân phối: $P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R),$
 $P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R),$

Lũy đẳng: $P \vee P = P, \quad P \wedge P = P,$

Hấp thụ: $P \vee (P \wedge Q) = P, \quad P \wedge (P \vee Q) = P,$

De Morgan: $\neg(P \vee Q) = \neg P \wedge \neg Q,$
 $\neg(P \wedge Q) = \neg P \vee \neg Q.$

Ngoài ra,

$$A \rightarrow B = \neg A \vee B,$$

và

$$A \leftrightarrow B = (\neg A \vee B) \wedge (A \vee \neg B) = (A \wedge B) \vee (\neg A \wedge \neg B).$$

2.2 Từ bảng chân lý đến công thức

Viết bảng chân lý từ một công thức là dễ. Chiều ngược lại cũng làm được bằng cách viết từ các dòng đúng hoặc từ các dòng sai. Ví dụ với hai biến P, Q , giả sử công thức A đúng ở các dòng (F, F), (F, T), (T, T), và sai ở (T, F). Từ các dòng đúng, ta viết:

$$A = (\neg P \wedge \neg Q) \vee (\neg P \wedge Q) \vee (P \wedge Q).$$

Từ dòng sai duy nhất, ta có thể viết gọn:

$$A = \neg P \vee Q.$$

2.3 Dạng chuẩn

Một **literal** là biến mệnh đề P hoặc phủ định của nó $\neg P$. **Mệnh đề hội** là hội của một số literal; **mệnh đề tuyển** là tuyển của một số literal.

Một công thức ở dạng **tuyển chuẩn** nếu nó là tuyển của các mệnh đề hội:

$$A_1 \vee A_2 \vee \cdots \vee A_n,$$

trong đó mỗi A_i là hội của các literal. Một công thức ở dạng **hội chuẩn** nếu nó là hội của các mệnh đề tuyển:

$$A_1 \wedge A_2 \wedge \cdots \wedge A_n,$$

trong đó mỗi A_i là tuyển của các literal.

Định lý dạng chuẩn nói rằng mọi công thức mệnh đề đều có một công thức tương đương ở dạng hội chuẩn và một công thức tương đương ở dạng tuyển chuẩn. Các luật như $A \rightarrow B = \neg A \vee B$, De Morgan và phân phối là công cụ cơ bản để biến đổi.

3 Bài toán SAT

3.1 Định nghĩa

SAT là bài toán quyết định xem một công thức logic có khả thỏa hay không. Thông thường công thức được đưa về dạng hội chuẩn:

$$A_1 \wedge A_2 \wedge \cdots \wedge A_n,$$

trong đó mỗi A_i là một mệnh đề tuyển

$$A_i = (P_{i1} \vee P_{i2} \vee \cdots \vee P_{im}).$$

3.2 2-SAT

2-SAT là trường hợp đặc biệt trong đó mỗi mệnh đề tuyển có không quá hai literal. Ví dụ:

$$(X_0 \vee X_2) \wedge (\neg X_0 \vee X_3) \wedge (X_1 \vee \neg X_3).$$

2-SAT có lời giải đa thức bằng đồ thị kéo theo. Với mỗi biến A , tạo hai đỉnh A và $\neg A$. Với mỗi mệnh đề $(A \vee B)$, dùng tương đương

$$A \vee B \equiv (\neg A \rightarrow B) \wedge (\neg B \rightarrow A),$$

nên thêm hai cung

$$\neg A \rightarrow B, \quad \neg B \rightarrow A.$$

Nếu trong đồ thị có cả đường đi $A \rightarrow^* \neg A$ và $\neg A \rightarrow^* A$, tức A và $\neg A$ nằm trong cùng một thành phần liên thông mạnh, thì công thức vô nghiệm. Ngược lại, thu gọn SCC thành DAG và chọn giá trị cho từng cặp đỉnh đối ngẫu theo thứ tự topo ngược để có một gán thỏa.

3.3 3-SAT và DPLL

Khi có mệnh đề tuyển chứa ba biến, cách làm trên không còn áp dụng trực tiếp. 3-SAT là bài toán NP-đầy đủ đầu tiên được biết đến, thông qua định lý Cook năm 1971. Với SAT tổng quát, ta thường phải tìm kiếm có cắt tỉa.

Thuật toán Davis–Putnam–Logemann–Loveland, gọi tắt là **DPLL**, được đưa ra năm 1962 như một cải tiến của thuật toán Davis–Putnam năm 1960. Trong nhiều thập niên, nó vẫn là lõi của nhiều bộ giải SAT hiệu quả.

Dạng khung:

```

function DPLL(Phi):
    if Phi is empty:
        return true
    if Phi contains an empty clause:
        return false

    while Phi has a unit clause l:
        set l to true and simplify Phi

    while Phi has a pure literal x:
        set x to true and simplify Phi

    choose an unassigned variable y
    return DPLL(Phi with y = true)
        or DPLL(Phi with y = false)

```

Ở đây Φ là tập các mệnh đề tuyến, được hiểu là hội của chúng. Hai cấu tử quan trọng là **unit propagation**, tức mệnh đề chỉ còn một literal thì literal đó buộc phải đúng, và **pure literal elimination**, tức nếu một biến chỉ xuất hiện với cùng một dấu thì có thể gán nó để làm đúng tất cả các mệnh đề chứa nó.

4 SMT

4.1 Từ SAT đến SMT

SAT chỉ xử lý ràng buộc trên biến Boolean. Trong nhiều bài toán, ràng buộc không chỉ là tuyến các biến Boolean, mà còn là phương trình hoặc bất phương trình trên số nguyên, số thực, mảng, hoặc bit-vector. **SMT**, viết tắt của *Satisfiability Modulo Theories*, là bài toán xét khả thỏa của công thức logic trên một lý thuyết nền cụ thể.

Một số miền thường gặp:

- Boolean, tức SAT cổ điển;
- số nguyên;
- số thực;
- mảng;
- bit-vector.

SMT có hai hướng nghiên cứu lớn. Một hướng gần với toán học: giải ràng buộc trên số nguyên, số thực, tuyến tính hoặc phi tuyến. Hướng còn lại gần với máy tính: bit-vector, mảng và các phép toán mô phỏng chính xác hành vi của chương trình.

4.2 Ví dụ giải hệ tuyến tính trên bit-vector

Xét hệ trên bit-vector độ rộng 3:

$$3x + 4y + 2z = 0,$$

$$2x + 2y + 2 = 0,$$

$$4y + 2x + 2z = 0.$$

Các phép toán được hiểu theo modulo $2^3 = 8$. Từ phương trình đầu, vì $3^{-1} \equiv 3 \pmod{8}$, có thể biểu diễn

$$x = 4y + 2z.$$

Thay vào hai phương trình còn lại:

$$2y + 4z + 2 = 0,$$

$$4y + 6z = 0.$$

Tất cả hệ số đều chẵn nên chia cho 2, đồng thời giảm độ rộng còn 2 bit và bỏ bit cao:

$$y[1:0] + 2z[1:0] + 1 = 0,$$

$$2y[1:0] + 3z[1:0] = 0.$$

Giải $y[1:0]$ theo $z[1:0]$:

$$y[1:0] = 2z[1:0] + 3.$$

Thay vào phương trình còn lại:

$$3z[1:0] + 2 = 0,$$

từ đó thu được

$$z[1:0] = 2, \quad y[1:0] = 3.$$

Quay lại độ rộng 3, viết $y = y' @ 2$, $z = z' @ 3$ theo cách ký hiệu nối bit trong slide, và

$$x = 4y + 2z.$$

Ví dụ này minh họa đặc trưng của bit-vector: phép chia, bỏ bit cao và đồng dư theo độ rộng đều là một phần của ngữ nghĩa.

5 Ứng dụng

5.1 Giải phương trình

SMT có thể được dùng trực tiếp để kiểm tra một hệ phương trình hoặc bất phương trình có nghiệm hay không, đồng thời trả về một nghiệm nếu tồn tại. Tùy miền, bộ giải sẽ dùng các thủ tục khác nhau: Gaussian elimination cho hệ tuyến tính, simplex cho ràng buộc tuyến tính trên số thực, hoặc các thủ tục bit-vector cho phép toán mức máy.

5.2 Phát hiện lỗi chương trình

Một ứng dụng quan trọng là quét lỗi chương trình bằng cách biến đường đi thực thi thành ràng buộc. Ví dụ:

```
int two_hop(int x) {
    int a[4] = {3, 0, 2, 1};
    if (x < 0 || x > 3) return -1;
    return a[a[x] - 1];
}
```

Điều kiện đầu vào bảo đảm $x \in [0, 3]$, nhưng khi $x = 1$ thì $a[x] = 0$, nên truy cập $a[a[x] - 1] = a[-1]$, vượt ngoài mảng. Một bộ giải SMT có thể sinh tự động đầu vào $x = 1$ bằng cách mô hình hóa mảng, phép so sánh và phép truy cập theo đường đi.

Tài liệu tham khảo

- Shi Chunyi, *Mathematical Logic and Set Theory*, Tsinghua University Press, 2000.
- Zhao Shuang, *Analysis of 2-SAT solving methods*.
- Wikipedia: Boolean satisfiability problem.
- Wikipedia: DPLL algorithm.
- Wikipedia: Satisfiability Modulo Theories.
- Cristian Cadar, Vijay Ganesh, Peter Pawlowski, David Dill, and Dawson Engler. *EXE: Automatically generating inputs of death*. ACM CCS, 2006.
- Vijay Ganesh. *Decision Procedures for Bit-vectors, Arrays and Integers*. September 2007.